

Building LLM Reasoners

Assignment 3

Reasoning RL

Siva Satvik

sm12779

3.2 math_baseline

- === Intellect Test (data/intellect_math/test) ===

```
correct_both1: 318
```

```
format1_answer0: 155
```

```
format0_answer0: 27
```

Intellect Accuracy: 0.6360

=== MATH Test ===

```
correct_both1: 304
```

```
format1_answer0: 166
```

```
format0_answer0: 30
```

MATH Accuracy: 0.6080

Looking at examples where the format is wrong, we see that the model often produces an answer without showing its work, which results in a format score of 0. In some cases, the model attempts to show its work but does so incorrectly by repeating the same steps, which also results in a format score of 0. Example of format 0 and answer 0:

```

"dataset": "math",
"category": "formato_answer0",
"format_reward": 0.0,
"answer_reward": 0.0,
"reward": 0.0,
"ground_truth": "4",
"parsed_answer": null,
"response": "
To solve this problem, we need to consider all possible ways of inserting parentheses into the expression  $((2 \times 3 \times 4 \times 5 + 1))$ . The expression can be evaluated in different ways depending on where the parentheses are placed. Let's break down the problem step-by-step.

 $((2 \times 3 \times 4 \times 5 + 1))$ ,  $((2 \times 3 \times 4) \times 5 + 1)$ ,  $((2 \times 3 \times 4) \times 5 + 1)$ , etc. We need to evaluate each of these possibilities.

 $((2 \times 3 \times 4 \times 5 + 1))$ ,  $((2 \times 3 \times 4) \times 5 + 1)$ ,  $((2 \times 3 \times 4) \times 5 + 1)$ , etc. We need to evaluate each of these possibilities.

 $((2 \times 3 \times 4 \times 5 + 1))$ ,  $((2 \times 3 \times 4) \times 5 + 1)$ ,  $((2 \times 3 \times 4) \times 5 + 1)$ , etc. We need to evaluate each of these possibilities.

Python code to evaluate all possible ways of inserting parentheses and count the number of distinct values obtained.

python
import itertools

# Define the expression components
ncomponents = [2, 3, 4, 5, 1]
# Function to evaluate the expression with given parentheses
def evaluate_expression(expr, parentheses):
    # Split the expression based on parentheses
    sub_exprs = expr.split('(')
    # Evaluate the sub-expressions recursively
    value = sub_exprs[0]
    for i in range(1, len(sub_exprs)):
        value = sub_exprs[i]
        if value == '(':
            value = evaluate_expression(value, parentheses)
        return value
# Generate all possible ways to insert parentheses
def generate_parentheses(expr):
    n = len(expr)
    for i in range(1, n):
        for j in range(i+1, n):
            for k in range(j+1, n):
                for l in range(k+1, n):
                    for m in range(l+1, n):
                        for n in range(m+1, n):
                            for o in range(n+1, n):
                                for p in range(o+1, n):
                                    for q in range(p+1, n):
                                        for r in range(q+1, n):
                                            for s in range(r+1, n):
                                                for t in range(s+1, n):
                                                    for u in range(t+1, n):
                                                        for v in range(u+1, n):
                                                            for w in range(v+1, n):
                                                                for x in range(w+1, n):
                                                                    for y in range(x+1, n):
                                                                        for z in range(y+1, n):
                                                                            parentheses = [expr[i], expr[i+1], expr[i+2], expr[i+3], expr[i+4], expr[i+5], expr[i+6], expr[i+7], expr[i+8], expr[i+9], expr[i+10], expr[i+11], expr[i+12], expr[i+13], expr[i+14], expr[i+15], expr[i+16], expr[i+17], expr[i+18], expr[i+19], expr[i+20], expr[i+21], expr[i+22], expr[i+23], expr[i+24], expr[i+25], expr[i+26], expr[i+27], expr[i+28], expr[i+29], expr[i+30], expr[i+31], expr[i+32], expr[i+33], expr[i+34], expr[i+35], expr[i+36], expr[i+37], expr[i+38], expr[i+39], expr[i+40], expr[i+41], expr[i+42], expr[i+43], expr[i+44], expr[i+45], expr[i+46], expr[i+47], expr[i+48], expr[i+49], expr[i+50], expr[i+51], expr[i+52], expr[i+53], expr[i+54], expr[i+55], expr[i+56], expr[i+57], expr[i+58], expr[i+59], expr[i+60], expr[i+61], expr[i+62], expr[i+63], expr[i+64], expr[i+65], expr[i+66], expr[i+67], expr[i+68], expr[i+69], expr[i+70], expr[i+71], expr[i+72], expr[i+73], expr[i+74], expr[i+75], expr[i+76], expr[i+77], expr[i+78], expr[i+79], expr[i+80], expr[i+81], expr[i+82], expr[i+83], expr[i+84], expr[i+85], expr[i+86], expr[i+87], expr[i+88], expr[i+89], expr[i+90], expr[i+91], expr[i+92], expr[i+93], expr[i+94], expr[i+95], expr[i+96], expr[i+97], expr[i+98], expr[i+99], expr[i+100], expr[i+101], expr[i+102], expr[i+103], expr[i+104], expr[i+105], expr[i+106], expr[i+107], expr[i+108], expr[i+109], expr[i+110], expr[i+111], expr[i+112], expr[i+113], expr[i+114], expr[i+115], expr[i+116], expr[i+117], expr[i+118], expr[i+119], expr[i+120], expr[i+121], expr[i+122], expr[i+123], expr[i+124], expr[i+125], expr[i+126], expr[i+127], expr[i+128], expr[i+129], expr[i+130], expr[i+131], expr[i+132], expr[i+133], expr[i+134], expr[i+135], expr[i+136], expr[i+137], expr[i+138], expr[i+139], expr[i+140], expr[i+141], expr[i+142], expr[i+143], expr[i+144], expr[i+145], expr[i+146], expr[i+147], expr[i+148], expr[i+149], expr[i+150], expr[i+151], expr[i+152], expr[i+153], expr[i+154], expr[i+155], expr[i+156], expr[i+157], expr[i+158], expr[i+159], expr[i+160], expr[i+161], expr[i+162], expr[i+163], expr[i+164], expr[i+165], expr[i+166], expr[i+167], expr[i+168], expr[i+169], expr[i+170], expr[i+171], expr[i+172], expr[i+173], expr[i+174], expr[i+175], expr[i+176], expr[i+177], expr[i+178], expr[i+179], expr[i+180], expr[i+181], expr[i+182], expr[i+183], expr[i+184], expr[i+185], expr[i+186], expr[i+187], expr[i+188], expr[i+189], expr[i+190], expr[i+191], expr[i+192], expr[i+193], expr[i+194], expr[i+195], expr[i+196], expr[i+197], expr[i+198], expr[i+199], expr[i+200], expr[i+201], expr[i+202], expr[i+203], expr[i+204], expr[i+205], expr[i+206],


"prompt": "
Solve the following math problem efficiently and clearly. Think carefully and step by step about your response and reason before providing a final response. Conclude your response with final_answer. I hope it is correct. final_answer is just the final number or text that solves the problem. If the question is a multiple choice question, final_answer should be the letter indicating your correct response (e.g. A or B). The expression  $((2 \times 3 \times 4 \times 5 + 1))$  is equal to 121, since multiplication is carried out before addition. However, we can obtain values other than 121 for this expression if we are allowed to change it by inserting parentheses. For example, we can obtain 144 by writing  $((2 \times 3 \times 4 \times 5 + 1))$ . In total, how many values can be obtained from the expression  $((2 \times 3 \times 4 \times 5 + 1))$  by inserting parentheses? (Note that rearranging terms is not allowed, only inserting parentheses)."
```

In the case of examples where the format is correct but the answer is wrong, we see that the model often follows a reasonable problem-solving process but makes a computational error at some point, leading to an incorrect final answer. In some cases, the model may also misinterpret the question or apply an incorrect mathematical principle, which can also lead to a wrong answer despite a correct format. Example of format 1 and answer 0:

```

"idx": 7,
"dataset": "math",
"category": "format1_answer0",
"format_reward": 1.0,
"answer_reward": 0.0,
"reward": 0.0,
"ground_truth": "90^\circ",
"parsed_answer": "100.30",
"response": "
To find the angle between the two lines, we need to determine the direction vectors of
the lines and then use the dot product formula to find the angle between these vectors.
Find the direction vectors of the lines:
For the first line, the equation is  $2x = 3y = -2z$ .
We can express this as  $x = t, y = \frac{2t}{3}, z = -\frac{2t}{3}$ . So, the direction vector of the first line is  $\mathbf{d_1} = (1, \frac{2}{3}, -\frac{2}{3})$ .
For the second line, the equation is  $6x = -y = -4z$ . We can express this as  $x = s, y = -6s, z = -\frac{s}{4}$ . So, the direction vector of the second line is  $\mathbf{d_2} = (1, -6, -\frac{1}{4})$ .
Normalize the direction vectors:
The magnitude of  $\mathbf{d_1}$  is  $\sqrt{1^2 + (\frac{2}{3})^2 + (-\frac{2}{3})^2} = \sqrt{1 + \frac{4}{9} + \frac{4}{9}} = \sqrt{\frac{17}{9}} = \frac{\sqrt{17}}{3}$ .
So, the unit vector in the direction of  $\mathbf{d_1}$  is  $\hat{\mathbf{d_1}} = (\frac{3}{\sqrt{17}}, \frac{2}{\sqrt{17}}, -\frac{2}{\sqrt{17}})$ .
The magnitude of  $\mathbf{d_2}$  is  $\sqrt{1^2 + (-6)^2 + (-\frac{1}{4})^2} = \sqrt{1 + 36 + \frac{1}{16}} = \sqrt{\frac{585}{16}} = \frac{3\sqrt{65}}{4}$ .
So, the unit vector in the direction of  $\mathbf{d_2}$  is  $\hat{\mathbf{d_2}} = (\frac{4}{3\sqrt{65}}, -\frac{24}{3\sqrt{65}}, -\frac{1}{3\sqrt{65}})$ .
Find the angle between the direction vectors:
The dot product of  $\hat{\mathbf{d_1}}$  and  $\hat{\mathbf{d_2}}$  is  $\frac{3}{\sqrt{17}} \cdot \frac{4}{3\sqrt{65}} + \frac{2}{\sqrt{17}} \cdot (-\frac{24}{3\sqrt{65}}) + (-\frac{2}{\sqrt{17}}) \cdot (-\frac{1}{3\sqrt{65}}) = \frac{4}{\sqrt{1055}} - \frac{16}{\sqrt{1055}} + \frac{2}{\sqrt{1055}} = -\frac{10}{\sqrt{1055}}$ .
The angle  $\theta$  between the two vectors is given by:  $\cos \theta = -\frac{10}{\sqrt{1055}}$ .
So,  $\theta = \arccos(-\frac{10}{\sqrt{1055}})$ .
Convert the angle to degrees:
Using Python to calculate the angle in degrees:
python
import math
theta_radians = -2 / math.sqrt(65)
theta_degrees = math.degrees(theta_radians)
print(theta_degrees)
Output: 100.304646876604
Therefore, the final answer is:  $100.30^\circ$ .
"
prompt: "Solve the following math problem efficiently and clearly. Think carefully and step by step about your response and reason before providing a final response. Conclude your response with: Therefore, the final answer is:  $\boxed{\text{answer}}$ . I hope it is correct. Where [answer] is just the final number or expression that solves the problem. If the question is a multiple choice question, [answer] should be the letter indicating your correct response (e.g. \text{A} or \text{B}). The set of points  $(x,y,z)$  that satisfy  $2x = 3y = -2z$  is a line. The set of points  $(x,y,z)$  that satisfy  $6x = -y = -4z$  is another line. Find the angle between these lines, in degrees."

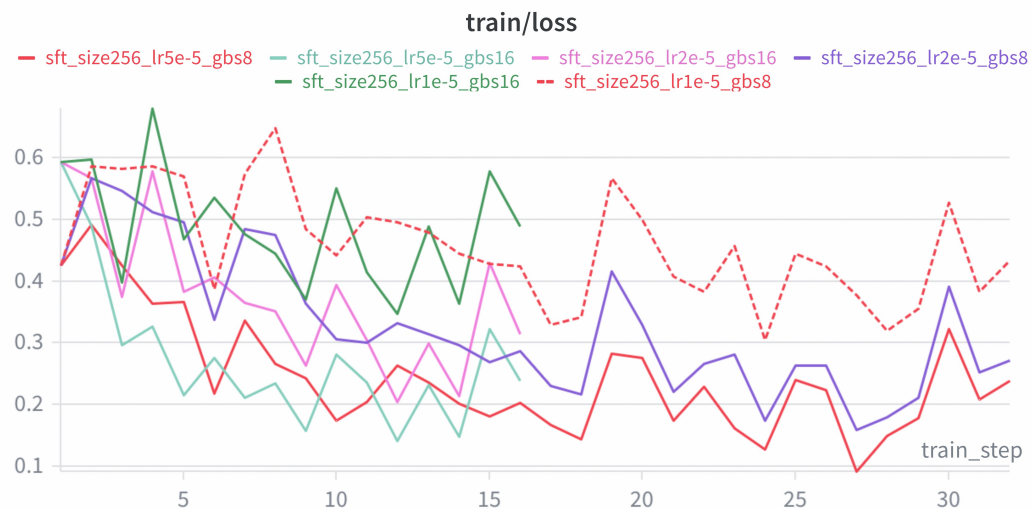
```

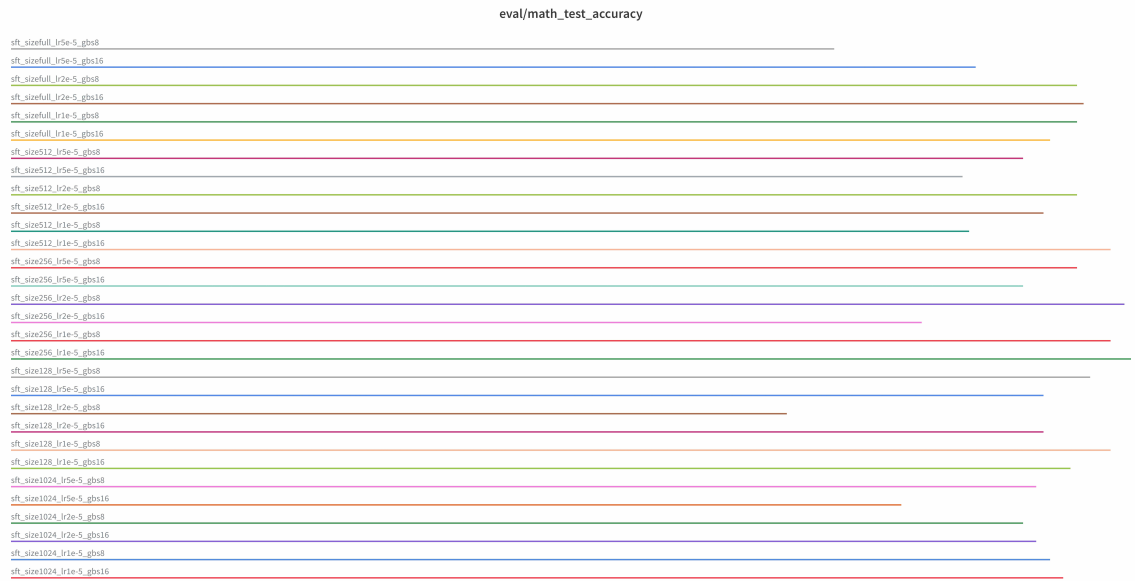
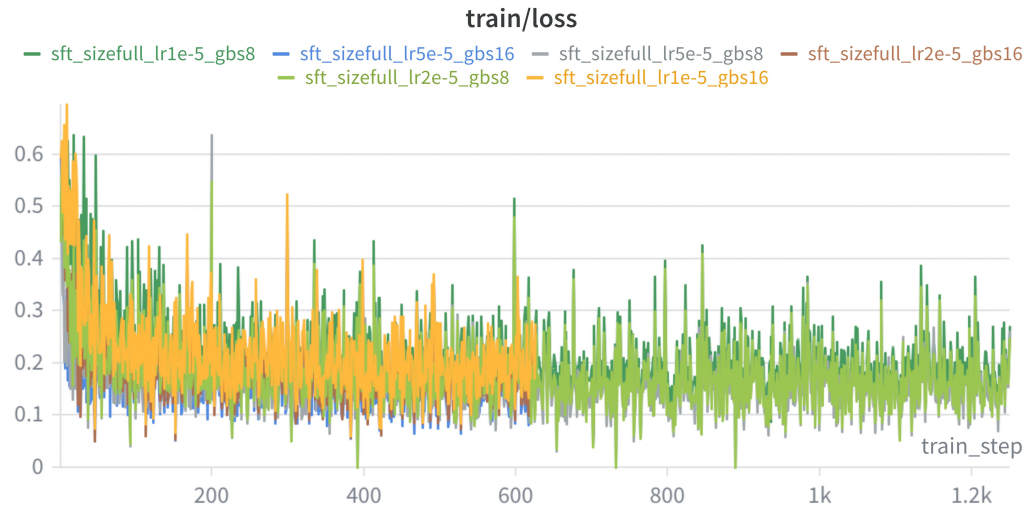
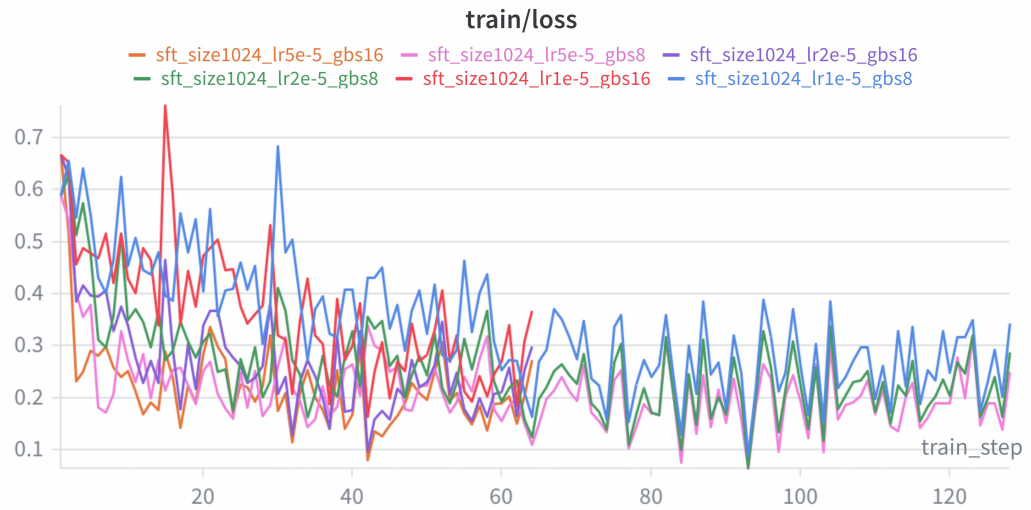
- (b) The accuracy is around 60% which is a good baseline for zero-shot performance. The evaluation metrics seem to be working well in distinguishing between correct and incorrect answers, as well as between correct and incorrect formats.

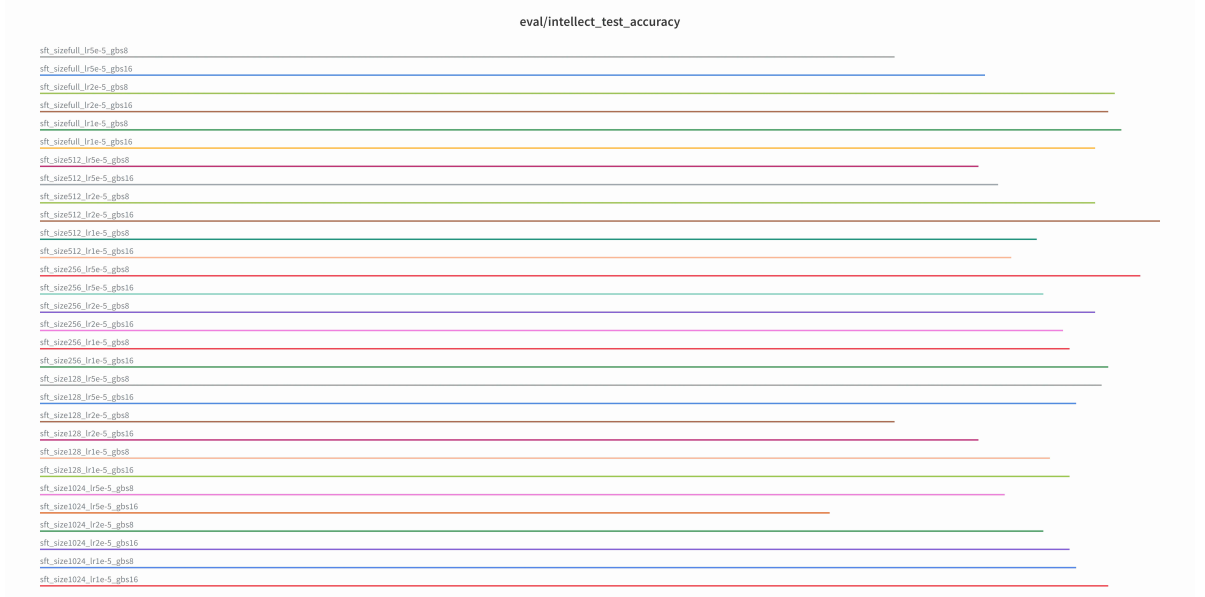
4 Supervised Finetuning for MATH

4.3 sft_experiment

- The training losses and validation accuracies for various dataset sizes, batch sizes and learning rates are given below.







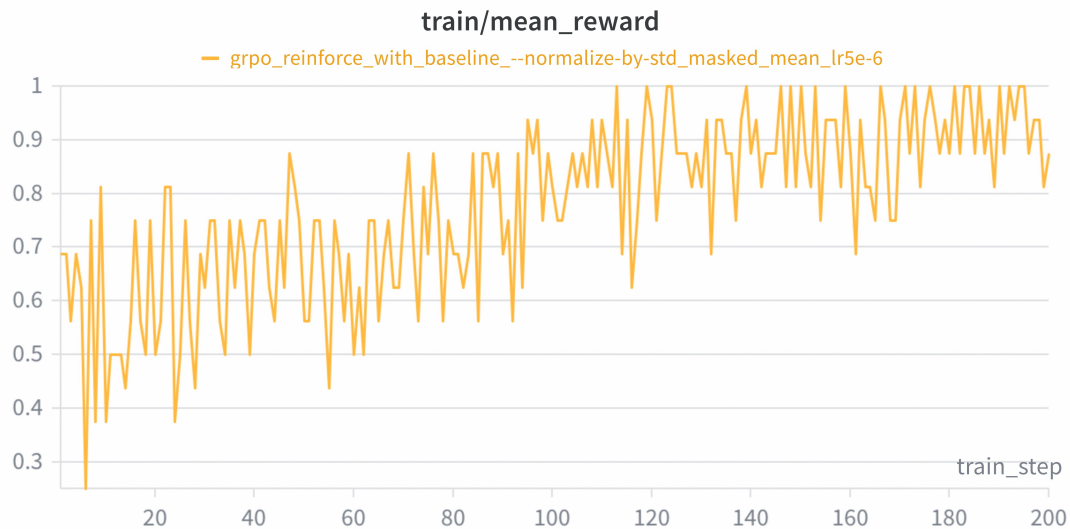
There are multiple runs where I saw 40% decrease in train loss like the run with dataset size 128, batch size 8 and learning rate $1e-5$. Validation accuracy for the best model for each run is also given in the graphs.

- ii. The best validation accuracy on the intellect test set is around **67.6%** (**dataset size: 512, learning rate: $2e-5$, global batch size: 16**) and the best validation accuracy on the MATH test set is around **64.8%** (**dataset size: 256, learning rate: $1e-5$, global batch size: 16**). The best validation accuracy on the intellect test set is slightly higher than the best validation accuracy on the MATH test set, which suggests that the model may be slightly better at generalizing to the intellect test set compared to the MATH test set. This could be due to differences in the distribution of questions or the difficulty level between the two test sets. It's also possible that the model has learned to perform better on certain types of questions that are more prevalent in the intellect test set compared to the MATH test set.

7 Group Relative Policy Optimization

7.2 gppo_train_loop

The figure below shows the reward vs training step curve for a GRPO experiment with a learning rate of $5e-6$. We can see that the reward increases steadily over the course of training, indicating that the model is learning to improve its performance on the task. The curve shows some fluctuations, which is expected in reinforcement learning due to the stochastic nature of the environment and the learning process. Overall, the trend suggests that the GRPO algorithm is effectively optimizing the policy to achieve higher rewards over time.



Example rollouts at every 50 steps over the course of 200 training steps are given below (the rollouts end just before `</answer>` tag)

```
{
  "train_step": 50,
  "examples": [
    {
      "prompt": " steps to reach the target number or it should be a single equation that
        ↪ results in the target.\nFor example:\nIf the list of numbers was [1, 2, 3] and the
        ↪ target was 1, you could write:\n<answer>\nStep 1: 1 + 2 = 3\nStep 2: 3 / 3 =
        ↪ 1\n</answer>\nor\n<answer>\n(1 + 2) / 3\n</answer>\nLet's think step by step.\n",
      "gt": "32",
      "rollout": "Step 1: Start with the numbers given: [12, 24, 44].\nStep 2: Divide 44 by 24,
        ↪ which gives us 1.83 (approximated).\nStep 3: Multiply the result by 12, which gives us
        ↪ 22.04 (approximated).\nStep 4: Subtract 24 from the result, which gives us -1.96
        ↪ (approximated).\nStep 5: Add 24 to the result, which gives us 22.04
        ↪ (approximated).\nStep 6: Divide the result by 2, which gives us 11.02
        ↪ (approximated).\nStep 7: Add 12 to the result, which gives us 23.02
        ↪ (approximated).\nStep 8: Subtract 24 from the result, which gives us -0.98
        ↪ (approximated).\nStep 9: Add 44 to the result, which gives us 43.02 (approximated",
      "reward": 0.0
    }
  ]
},
{
  "train_step": 100,
  "examples": [
    {
      "prompt": " steps to reach the target number or it should be a single equation that
        ↪ results in the target.\nFor example:\nIf the list of numbers was [1, 2, 3] and the
        ↪ target was 1, you could write:\n<answer>\nStep 1: 1 + 2 = 3\nStep 2: 3 / 3 =
        ↪ 1\n</answer>\nor\n<answer>\n(1 + 2) / 3\n</answer>\nLet's think step by step.\n",
      "gt": "84",
```

```

        "rollout": "Step 1: 22 + 15 = 37\nStep 2: 37 + 77 = 114\nStep 3: 114 - 95 = 19\nStep 4: 19  

        ↪ + 15 = 34\nStep 5: 34 + 50 = 84\nSo, the final answer is:\n<answer>\nStep 1: 22 + 15 =  

        ↪ 37\nStep 2: 37 + 77 = 114\nStep 3: 114 - 95 = 19\nStep 4: 19 + 15 = 34\nStep 5: 34 +  

        ↪ 50 = 84\n",
        "reward": 1.0
    }
]
}
{
    "train_step": 150,
    "examples": [
        {
            "prompt": " steps to reach the target number or it should be a single equation that  

            ↪ results in the target.\nFor example:\nIf the list of numbers was [1, 2, 3] and the  

            ↪ target was 1, you could write:\n<answer>\nStep 1: 1 + 2 = 3\nStep 2: 3 / 3 =  

            ↪ 1\n</answer>\nor\n<answer>\n(1 + 2) / 3\n</answer>\nLet's think step by step.\n",
            "gt": "46",
            "rollout": "Step 1: 28 + 33 = 61\nStep 2: 61 - 41 = 20\nStep 3: 20 + 26 = 46\nThe final  

            ↪ answer is 46.\nTherefore, the answer is:\n<answer>\nStep 1: 28 + 33 = 61\nStep 2: 61 -  

            ↪ 41 = 20\nStep 3: 20 + 26 = 46\n",
            "reward": 1.0
        }
    ]
}
{
    "train_step": 200,
    "examples": [
        {
            "prompt": " steps to reach the target number or it should be a single equation that  

            ↪ results in the target.\nFor example:\nIf the list of numbers was [1, 2, 3] and the  

            ↪ target was 1, you could write:\n<answer>\nStep 1: 1 + 2 = 3\nStep 2: 3 / 3 =  

            ↪ 1\n</answer>\nor\n<answer>\n(1 + 2) / 3\n</answer>\nLet's think step by step.\n",
            "gt": "70",
            "rollout": "Step 1: 65 - 23 = 42\nStep 2: 92 - 42 = 50\nStep 3: 50 + 20 = 70\nSo the  

            ↪ answer is:\n<answer>\nStep 1: 65 - 23 = 42\nStep 2: 92 - 42 = 50\nStep 3: 50 + 20 =  

            ↪ 70\n",
            "reward": 1.0
        }
    ]
}

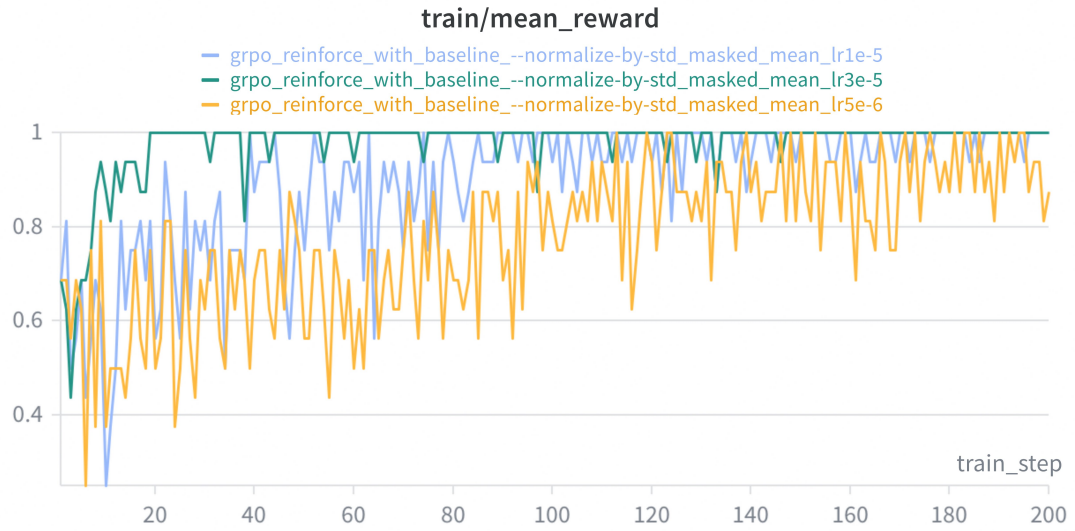
```

8 GRPO Experiments

8.1.1 grpo_learning_rate

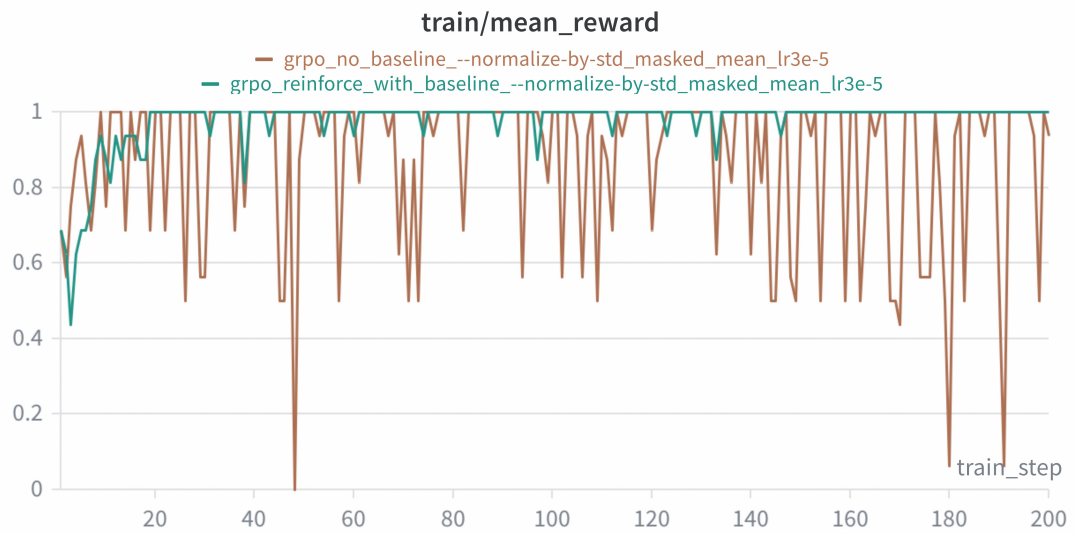
The figure below shows the reward vs training step curve for GRPO experiments with learning rates of $5e-6$, $1e-5$, and $3e-5$. We can see that the reward increases steadily over the course of training for all three learning rates, indicating that the model is learning to improve its performance on the task. The curve for the learning rate of $3e-5$ shows a slightly faster increase in reward compared to the other two learning rates, suggesting that it may be more effective for this particular task. Learning rate of

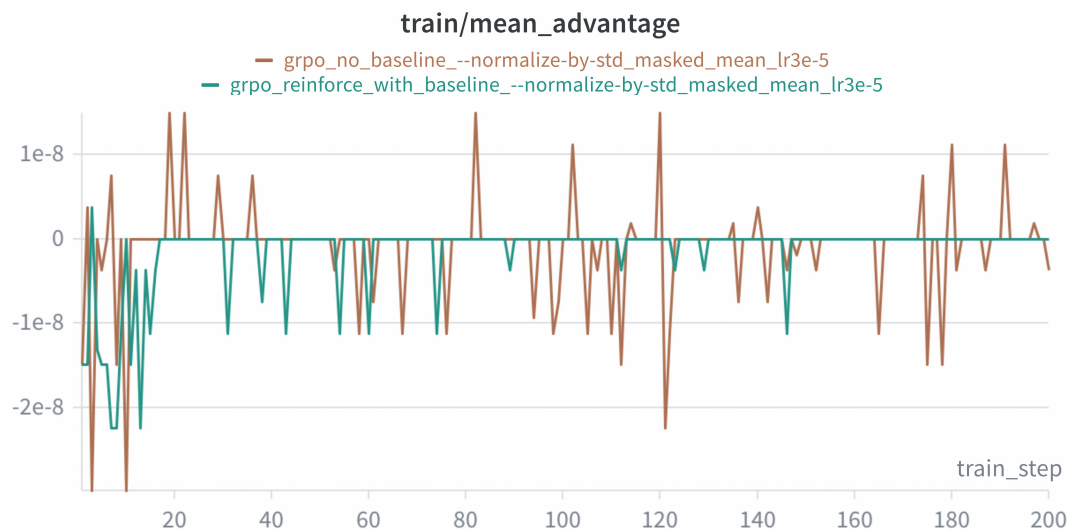
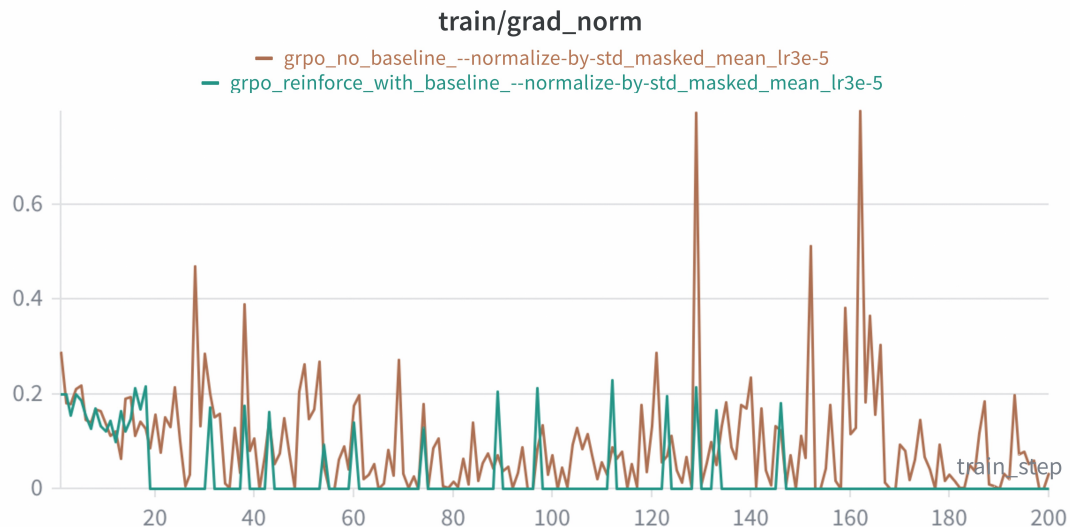
5e-6 shows a slower increase in reward, which may indicate that it is too low for effective learning.



8.1.2 grpo_baseline

The figures below show differences in `reinforce_with_baseline` and `no_baseline` runs.





We can see that the reward increase for the **no_baseline** run is much more unstable compared to the **reinforce_with_baseline** run, which shows a steady increase in reward over time. The gradient norm for the **no_baseline** run is also much higher and more variable compared to the **reinforce_with_baseline** run, which has a lower and more stable gradient norm. The advantage values for the **no_baseline** run are also much more variable and can take on much larger values compared to the **reinforce_with_baseline** run, which has smaller and more stable advantage values. Overall, these observations suggest that using a baseline in the REINFORCE algorithm can help stabilize training and lead to better performance.

8.1.3 think_about_length_normalization

masked_mean averages over each sequence's active tokens, so every completion contributes roughly equally regardless of length; this is usually more stable and reduces bias toward long responses, but it can under-credit long trajectories by shrinking their per-token signal.

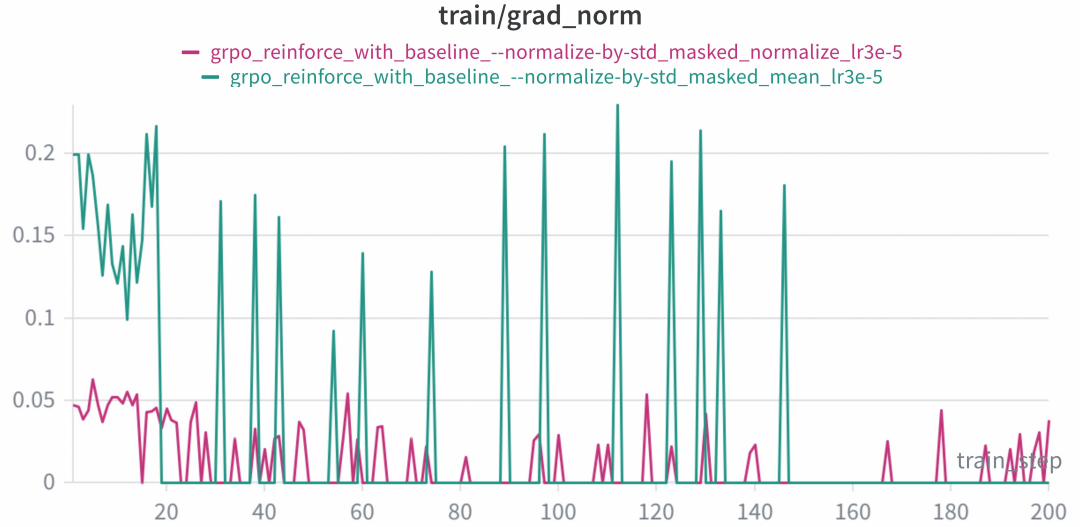
masked_normalize sums over active tokens and divides by a fixed constant, which keeps per-token scaling more uniform across samples and can preserve stronger token-level credit assignment, but it

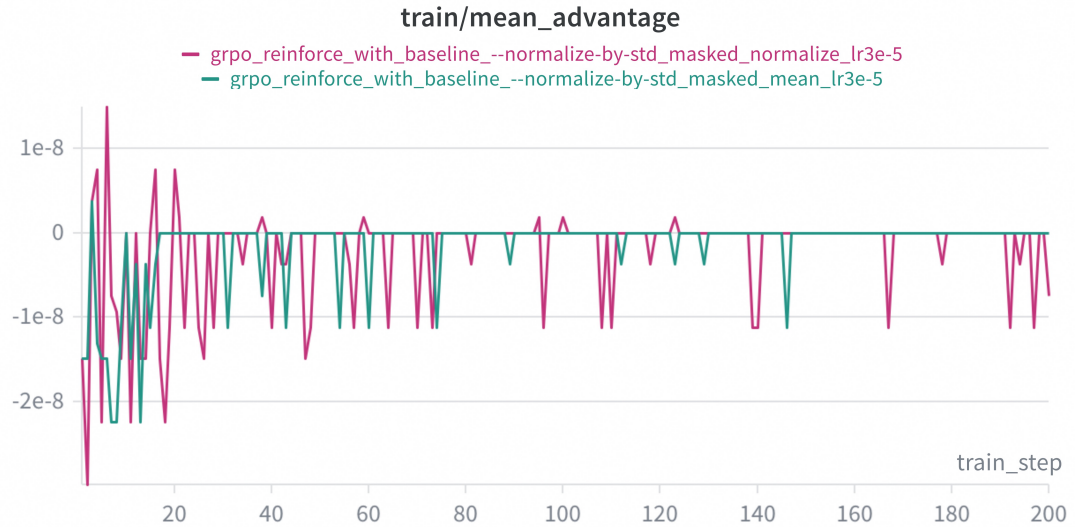
tends to make total update magnitude more length-dependent and can increase optimization volatility.

In settings with high length variation or where long chains of reasoning should receive proportionally more total credit, `masked_normalize` can be preferable, while in tasks like Countdown (shorter outputs, equal-example weighting) `masked_mean` is often the safer default.

8.1.4 grpo_length_normalization

The figures below show differences in `masked_mean` and `masked_normalize` normalization runs.

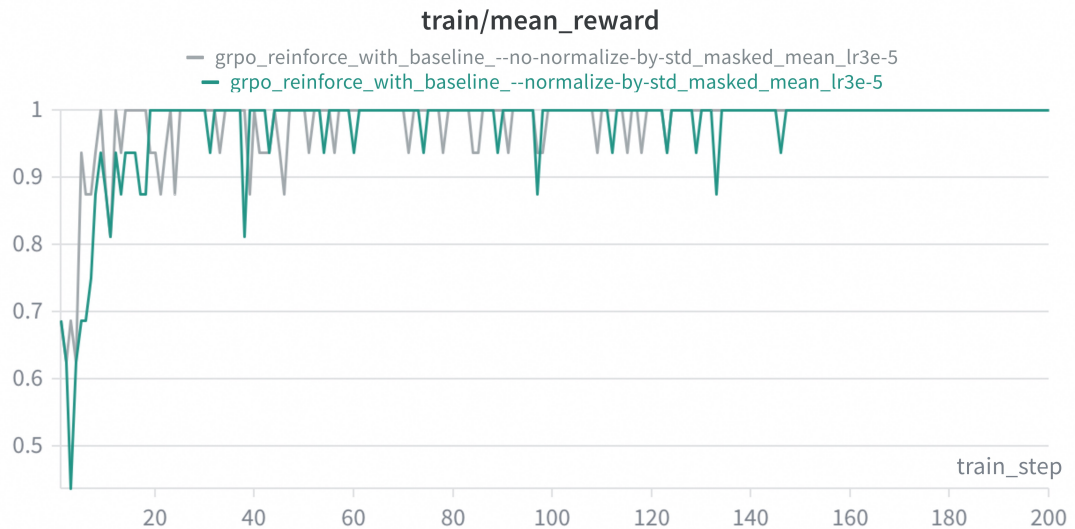


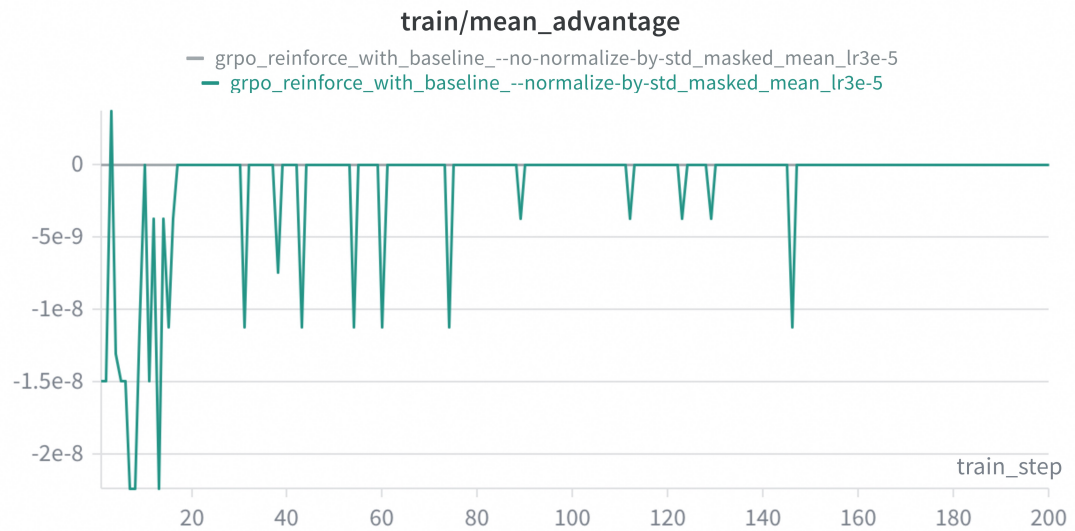
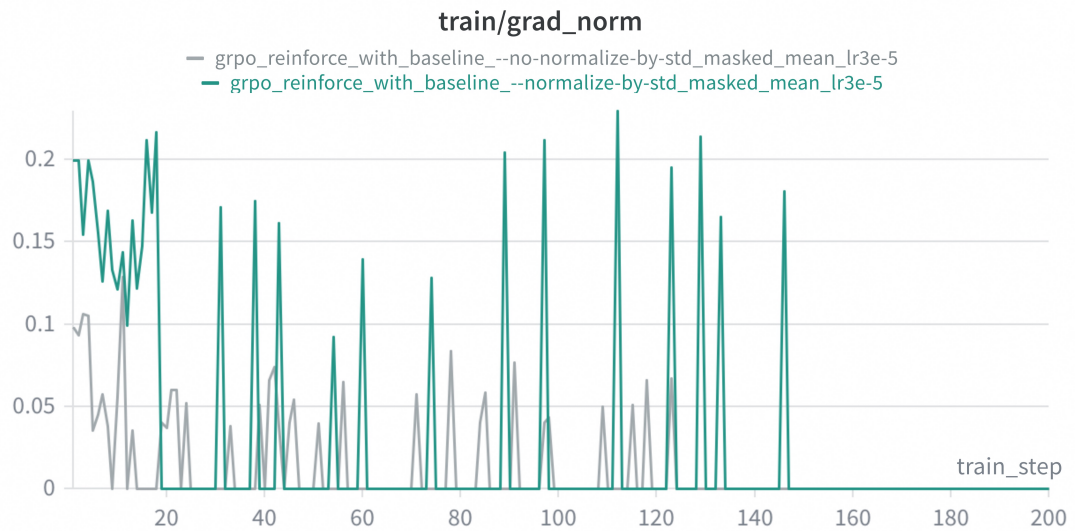


We can see that the reward increase for the `masked_mean` and `masked_normalize` runs are mostly similar but the `masked_mean` run shows a slightly faster increase and stability. The gradient norm for the `masked_normalize` run seems low compared to the `masked_mean` run, but the number of variations across training is higher for the `masked_normalize` run. The `masked_mean` run had a higher overall gradient norm level, while `masked_normalize` showed more frequent short spikes and a few late-training reward spikes, which suggests it is somewhat more sensitive to batch-to-batch variation. The advantage values for the `masked_normalize` normalization run are more variable and can take on larger values compared to the `masked_mean` run, especially towards the end of training.

8.1.5 grpo_group_standard_deviation

The figures below show differences in `use_std_normalization == True` and `use_std_normalization == False` runs.





We can see that the reward increase for the normalized-by-std and no-normalize-by-std runs are mostly similar. Compared to the normalized-by-std run, the no-normalize-by-std run shows consistently lower gradient norms, while the spike frequency is broadly similar, suggesting slightly more stability for the no-normalize-by-std run. The mean advantage stays at approximately zero throughout the no-normalize run, which is expected from mean-centering without standard-deviation scaling. Despite these optimization-scale differences, the reward curves are largely similar, suggesting comparable task-level performance on Countdown.